

Project Optimization Report

Faster, More Reliable, and More Scalable

1. Executive Summary

This project is a modern Next.js application using React 19, Supabase, Sentry, and VideoSDK. It already has a strong base with Docker support, standalone build output, health checks, and image optimization. The optimization goal is to improve speed, reliability, and scalability without redesigning the whole system.

2. Current Strengths

The application already has important production foundations: Node 22 setup, Next.js 16, React 19, standalone deployment, non-root container user, health checks, and image optimization. These features reduce many common production risks and show the project is already aligned with modern deployment practices.

3. Performance Optimization

To make the application faster, prioritize server-side rendering for data-heavy pages, route-based code splitting, lazy loading of dashboards and media-heavy components, and better caching for datasets such as categories, mentor listings, and reusable metadata. Reduce large API payloads and avoid fetching unnecessary fields from Supabase.

4. Reliability Optimization

Improve reliability by adding structured logging, Sentry alerts, retries for third-party services, timeout handling, graceful failure states, and better readiness/liveness checks. Keep build validation on CI with lint, type check, and production build verification before deployment.

5. Scalability Optimization

For scalability, run multiple app replicas behind a load balancer, use a CDN for static content, add Redis cache for repeated reads, and move long-running tasks into background workers. Database growth should be handled through indexing, pagination, and query limits.

6. Kubernetes and Docker Readiness

This project is already close to production containerization. To operate more like a Kubernetes-based deployment, add Deployment, Service, Ingress, HPA, Secrets, resource limits, probes, and centralized observability. Container health checks and standalone server output are already good foundations.

7. Recommended Action Plan

Phase 1: optimize frontend performance and caching. Phase 2: add monitoring, health checks, and runtime resilience. Phase 3: add orchestration, autoscaling, and background processing. This staged plan reduces risk while improving the app gradually.

8. Expected Outcomes

After implementation, the project will deliver faster page loads, reduced database pressure, better resilience under failures, improved app stability, and easier horizontal scaling for more users and traffic.

9. Conclusion

The project is already in a strong modern state. The right next move is not a rewrite, but controlled optimization across performance, reliability, and scalability. This layered approach creates a production-ready architecture that is efficient, stable, and ready to scale.

Prepared for: ConnectIQO Web Portal